Relatório de Continuidade — Dashboard Freelancer PreJud

Objetivo desta sessão: finalizar a integração do dashboard do freelancer com dados reais, mantendo o manual como referência obrigatória, sem quebrar fluxo, sem refatoração ampla e com foco no reaproveitamento da página pública do acordo com timeline e hash.

Base encontrada | Parou na Parte 3 | Arquivo da dashboard ficou incompleto/corrompido | Próximo chat deve retomar com 1 comando por vez

1) O que foi identificado e confirmado

- A rota pública do acordo existe em **app\p\[protocol]\page.tsx**.
- Essa página pública já usa **protocol** e **token (?t=...)**, carrega o acordo via serviço, exibe **timeline** e foi tratada como a visualização formal correta do acordo.
- O serviço **firebaseAgreementService.ts** já possui a lógica para buscar acordo por protocolo/ID e processar confirmação/contestação pública.
- O hook **useFreelancerDashboard.ts** já consome o serviço de dashboard e alimenta a tela principal.
- A Cloud Function **functions\src\index.ts** retorna os agreements com `...doc.data()`, portanto tende a trazer **protocol** e **clientAccessToken** quando esses campos estiverem gravados no Firestore.
- O problema funcional central era que o dashboard estava apontando para rota interna, enquanto o correto era abrir a rota pública **/p/{protocol}?t={clientAccessToken}** quando esses campos existirem.

2) Arquivos analisados e ajustes feitos / tentados

Arquivo	O que foi verificado / alterado	Motivo	Status real
`frontend\app\p\[protocol]\page.tsx`	Foi localizado e lido. Confirmado uso de protocolo, token, timeline, hash/estrutura formal do acordo.	Reaproveitar a visualização formal já existente.	Analisado

Arquivo	O que foi verificado / alterado	Motivo	Status real
`frontend\types\agreement.ts`	Foi conferido para validar campos necessários, inclusive `protocol`, `clientAccessToken`, `timeline`, `hash` e nomes exibidos na UI.	Garantir compatibilidade de tipos com a página pública e o dashboard.	Analisado
`frontend\services\firebaseAgreementService.ts`	Foi conferido e entendido como fonte da busca por protocolo/ID e confirmação/contestação pública.	Validar que a página pública já tem backend de leitura/ação.	Analisado
`frontend\hooks\useFreelancerDashboard.ts`	Foi lido para entender a alimentação do dashboard pelo serviço.	Localizar a camada correta para integração sem quebrar arquitetura.	Analisado
`frontend\services\dashboardService.ts`	Foi sobrescrito com uma versão nova para preservar `protocol` e `clientAccessToken` ao mapear agreements vindos da Cloud Function.	Permitir construção do link público no dashboard.	Alterado — precisa validação final em build/runtime
`functions\src\index.ts`	Foi apenas inspecionado. Confirmado retorno de agreements via `...doc.data()`.	Checar se backend já devolve os campos necessários.	Analisado, sem alteração nesta sessão
`frontend\app\(dashboard)\dashboard\page.tsx`	Foi substituído por conteúdo novo para integrar dados reais, remover duplicação visual e apontar botões para rota pública.	Finalizar a tela principal do freelancer.	Arquivo ficou incompleto / com problema de codificação / montagem parcial

3) Problemas reais ocorridos nesta continuidade

- Foi removido backup temporário do arquivo da dashboard antes da validação final.
- O arquivo **page.tsx** da dashboard sofreu erro de codificação UTF-8 durante tentativa de escrita automática.

- A build acusou: **failed to convert rope into string / invalid utf-8 sequence** no arquivo da dashboard.
- Também apareceu erro de importação do hook de autenticação: primeiro foi usado `@/hooks/useAuth`, depois identificado que o projeto pode usar `@/contexts/AuthContext`.
- O arquivo final não foi fechado com segurança: a resposta terminou no meio da **Parte 3**, deixando a dashboard sem conclusão confiável.

Conclusão honesta: a sessão avançou no diagnóstico e no desenho da integração, mas a finalização da dashboard **não ficou pronta**. O ponto correto de retomada é reconstruir o arquivo `frontend\app\(dashboard)\dashboard\page.tsx` de forma íntegra, em UTF-8 e com importações corretas, antes de qualquer novo ajuste visual.

4) Estado em que o projeto parou

- **Parte concluída:** descoberta da rota pública correta, confirmação dos campos necessários e alinhamento da estratégia de link público.
- **Parte sensível:** o arquivo `dashboard/page.tsx` precisa ser refeito por completo e salvo corretamente.
- **Parte 3 mencionada no histórico:** a montagem do componente travou no meio da renderização do header/hero e não foi concluída.
- **Prioridade absoluta do próximo chat:** abrir o arquivo atual, verificar se está truncado/inválido, reconstruir inteiro e só depois validar build.

5) Instrução pronta para o próximo chat

```
Você vai continuar EXATAMENTE de onde parou a finalização do dashboard do
freelancer do projeto PreJud.

REGRAS OBRIGATÓRIAS:
- seguir o manual técnico já adotado
- NÃO revisar arquitetura global
- NÃO sugerir mudanças paralelas
- NÃO apagar, quebrar, improvisar ou reescrever sem ler o arquivo atual primeiro
- Windows 11 + PowerShell + VS Code
- 1 comando por vez
- sempre esperar o retorno do comando antes do próximo
- foco exclusivo em finalizar o arquivo frontend\app\
(dashboard)\dashboard\page.tsx
- objetivo funcional: dashboard deve abrir o acordo pela rota pública
/p/{protocol}?t={clientAccessToken} quando houver protocol e token; se não
houver, manter fallback interno
- preservar o layout contemporâneo já iniciado, mas priorizar funcionamento sem
quebrar

CONTEXTO REAL DO PONTO DE PARADA:
1. A rota pública correta do acordo já foi confirmada em app\p\
```

```
[protocol]\page.tsx.
2. O tipo Agreement e o firebaseAgreementService já foram conferidos.
3. O dashboardService.ts foi alterado para preservar protocol e
clientAccessToken, mas ainda precisa validação posterior.
4. O arquivo frontend\app\(dashboard)\dashboard\page.tsx ficou
incompleto/corrompido e a build acusou erro de UTF-8.
5. A sessão parou no meio da PARTE 3, com o componente da dashboard incompleto.

MISSÃO DO PRÓXIMO CHAT:
- abrir o arquivo atual da dashboard
- conferir se está truncado ou inválido
- devolver o arquivo COMPLETO e íntegro para colar no VS Code
- garantir import correto do auth context/hook existente no projeto
- garantir import do Loader2
- fechar corretamente todo o JSX
- só depois validar com build ou typecheck

IMPORTANTE:
- trabalhar com 1 comando por vez no PowerShell
- não usar &&
- preferir -LiteralPath em caminhos com parênteses ou colchetes
```

6) Sequência operacional sugerida para o próximo chat

Abaixo está a ordem prática para o próximo chat conduzir, sempre um comando por vez.

Etapa	Comando PowerShell Windows 11	Finalidade
1	`Get-Content -LiteralPath "C:\prejud-saas-new\frontend\app\(dashboard)\dashboard\page.tsx" | Select-Object -First 80`	Abrir o início do arquivo atual e verificar importações, codificação aparente e estrutura.
2	`Get-Content -LiteralPath "C:\prejud-saas-new\frontend\app\(dashboard)\dashboard\page.tsx" | Select-Object -Last 80`	Confirmar se o arquivo termina corretamente ou se está truncado.
3	`Get-ChildItem -Path "C:\prejud-saas-new\frontend" -Recurse -File | Where-Object { $_.Name -match "AuthContext|useAuth" } | Select-Object FullName`	Descobrir o caminho real do provider/hook de autenticação antes de montar os imports finais.
4	`Get-Content -LiteralPath "C:\prejud-saas-new\frontend\services\dashboardService.ts" | Select-Object -First 120`	Revalidar a alteração do service e checar se protocol/token estão preservados no mapeamento.
5	`Get-Content -LiteralPath "C:\prejud-saas-new\frontend\app\p\`[protocol`]\page.tsx" | Select-Object -`	Reconfirmar os campos efetivamente usados pela

Etapa	Comando PowerShell Windows 11	Finalidade
	`First 120`	página pública.
6	`npm run build 2>&1 | Select-Object -First 80`	Somente depois da correção completa do arquivo, validar a build.

7) Resumo executivo

O trabalho útil desta sessão foi o **diagnóstico correto da integração**: a dashboard do freelancer deve reaproveitar a visualização pública do acordo, usando **protocol + clientAccessToken**, e a base técnica para isso já foi localizada. O que ficou pendente foi a **finalização segura do arquivo da dashboard**, que parou no meio da Parte 3 e ficou inválido para build. O próximo chat não deve reabrir discussão de estratégia; deve apenas retomar a execução técnica, um comando por vez, até devolver o arquivo completo e íntegro.